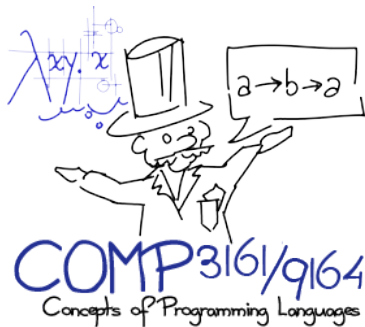




Imperative Programming Languages

Johannes Åman Pohjola
UNSW
Term 3 2023

Imperative Programming

imperō

Definition

Imperative programming is where programs are described as a series of *statements* or commands to manipulate mutable *state* or cause externally observable *effects*.

States may take the form of a *mapping* from variable names to their values, or even a model of a CPU state with a memory model (for example, in an *assembly language*).

The Old Days



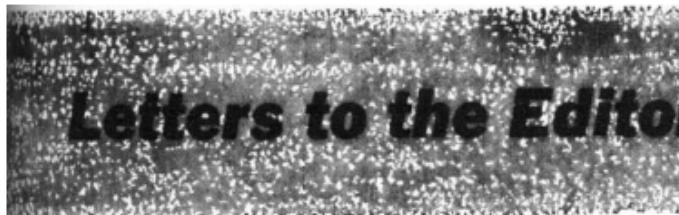
Early microcomputer languages used a line numbering system with GO TO statements used to arrange control flow.

Factorial Example in BASIC (1964)

```

1000 IF I = N THEN GOTO 100
1000 PRINT M
1000 GO TO 40
1000 END
```

Dijkstra (1968)



Go To Statement Considered Harmful

Key Words and Phrases: go to statement, jump instruction, branch instruction, conditional clause, alternative clause, repetitive clause, program intelligibility, program sequencing

CR Categories: 4.22. 5.23. 5.24

dyna
call
we c
text
dyna

The *structured programming* movement brought in *control structures* to mainstream use, such as conditionals and loops.

Factorial Example in Pascal (1970)

```
program factorial;  
var n : integer;  
    m : integer;  
    i : integer;  
begin  
  n := 5;  
  m := 1;  
  i := 0;  
  while (i < n) do  
  begin  
    i := i + 1;  
    m := m * i;  
  end;  
  println(m);
```

Syntax

We're going to specify a language **TinyImp**, based on **structured programming**. The syntax consists of **statements** and **expressions**.

Grammar

Stmt	::=	skip	<i>Do nothing</i>
		x := Expr	<i>Assignment</i>
		var y · Stmt	<i>Declaration</i>
		if Expr then Stmt else Stmt fi	<i>Conditional</i>
		while Expr do Stmt od	<i>Loop</i>
		Stmt ; Stmt	<i>Sequencing</i>
Expr	::=	<i>⟨Arithmetic expressions⟩</i>	

We already know how to make unambiguous **abstract syntax**, so we will use **concrete syntax** in the rules for readability.

Examples

Example (Factorial and Fibonacci)

```
var  $i$  ·  
var  $m$  ·  
 $i := 0$ ;  
 $m := 1$ ;  
while  $i < N$  do  
   $i := i + 1$ ;  
   $m := m \times i$   
od
```

```
var  $m$  · var  $n$  · var  $i$  ·  
 $m := 1$ ;  $n := 1$ ;  
 $i := 1$ ;  
while  $i < N$  do  
  var  $t$  ·  $t := m$ ;  
   $m := n$ ;  
   $n := m + t$ ;  
   $i := i + 1$   
od
```

Static Semantics

Types?

Static Semantics

Types? We only have one type (`int`), so type checking is a wash.

Scopes?

Static Semantics

Types? We only have one type (`int`), so type checking is a wash.

Scopes? We have to check that variables are declared before use.

Anything Else?

Static Semantics

Types? We only have one type (`int`), so type checking is a wash.

Scopes? We have to check that variables are declared before use.

Anything Else? We have to check that variables are *initialized* before they are used!

Static Semantics

Types? We only have one type (`int`), so type checking is a wash.

Scopes? We have to check that variables are declared before use.

Anything Else? We have to check that variables are *initialized* before they are used!

$$U; V \vdash s \text{ ok} \rightsquigarrow W$$

Note: $V \subseteq U$

Static Semantics

Types? We only have one type (`int`), so type checking is a wash.

Scopes? We have to check that variables are declared before use.

Anything Else? We have to check that variables are *initialized* before they are used!

$$U; V \vdash s \text{ ok} \rightsquigarrow W$$

Set of declared *free* variables



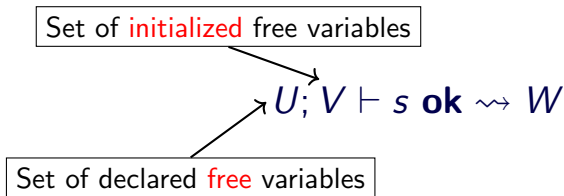
Note: $V \subseteq U$

Static Semantics

Types? We only have one type (`int`), so type checking is a wash.

Scopes? We have to check that variables are declared before use.

Anything Else? We have to check that variables are *initialized* before they are used!



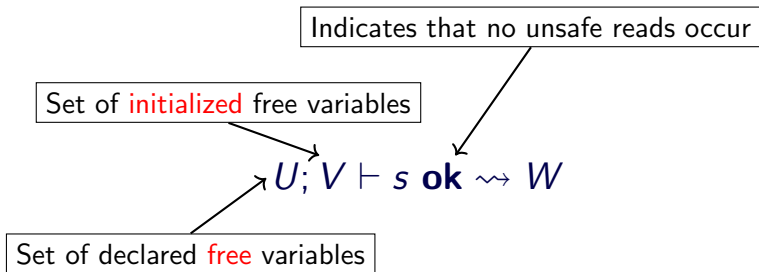
Note: $V \subseteq U$

Static Semantics

Types? We only have one type (`int`), so type checking is a wash.

Scopes? We have to check that variables are declared before use.

Anything Else? We have to check that variables are *initialized* before they are used!



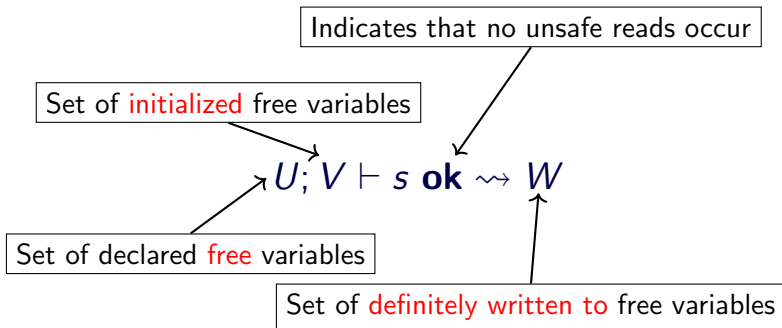
Note: $V \subseteq U$

Static Semantics

Types? We only have one type (`int`), so type checking is a wash.

Scopes? We have to check that variables are declared before use.

Anything Else? We have to check that variables are *initialized* before they are used!



Note: $V \subseteq U$

Static Semantics Rules

$$\frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset}$$

Static Semantics Rules

$$\frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow}$$

Static Semantics Rules

$$\frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow}$$

Static Semantics Rules

$$\frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow}$$

Static Semantics Rules

$$\frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}}$$

Static Semantics Rules

$$\frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}}$$

$$\frac{}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow}$$

Static Semantics Rules

$$\frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}}$$
$$\frac{U \cup \{y\}; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow}$$

Static Semantics Rules

$$\frac{}{U; V \vdash \text{skip} \text{ ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \text{ ok} \rightsquigarrow \{x\}}$$
$$\frac{U \cup \{y\}; V \vdash s \text{ ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \text{ ok} \rightsquigarrow W \setminus \{y\}}$$

Static Semantics Rules

$$\frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}}$$

$$\frac{U \cup \{y\}; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow W \setminus \{y\}}$$

$$U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \mathbf{ok} \rightsquigarrow$$

Static Semantics Rules

$$\frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}}$$

$$\frac{U \cup \{y\}; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow W \setminus \{y\}}$$

$$\frac{\text{FV}(e) \subseteq V}{U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \mathbf{ok} \rightsquigarrow}$$

Static Semantics Rules

$$\begin{array}{c} \frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}} \\[10pt] \frac{U \cup \{y\}; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow W \setminus \{y\}} \\[10pt] \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s_1 \mathbf{ok} \rightsquigarrow W_1}{U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \mathbf{ok} \rightsquigarrow} \end{array}$$

Static Semantics Rules

$$\begin{array}{c} \frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}} \\[10pt] \frac{U \cup \{y\}; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow W \setminus \{y\}} \\[10pt] \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s_1 \mathbf{ok} \rightsquigarrow W_1 \quad U; V \vdash s_2 \mathbf{ok} \rightsquigarrow W_2}{U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \mathbf{ok} \rightsquigarrow} \end{array}$$

Static Semantics Rules

$$\begin{array}{c}
 \frac{}{U; V \vdash \text{skip} \text{ ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \text{ ok} \rightsquigarrow \{x\}} \\
 \frac{U \cup \{y\}; V \vdash s \text{ ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \text{ ok} \rightsquigarrow W \setminus \{y\}} \\
 \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s_1 \text{ ok} \rightsquigarrow W_1 \quad U; V \vdash s_2 \text{ ok} \rightsquigarrow W_2}{U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi} \text{ ok} \rightsquigarrow W_1 \cap W_2}
 \end{array}$$

Static Semantics Rules

$$\begin{array}{c} \frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}} \\[10pt] \frac{U \cup \{y\}; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow W \setminus \{y\}} \\[10pt] \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s_1 \mathbf{ok} \rightsquigarrow W_1 \quad U; V \vdash s_2 \mathbf{ok} \rightsquigarrow W_2}{U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \mathbf{ok} \rightsquigarrow W_1 \cap W_2} \\[10pt] \frac{}{U; V \vdash \text{while } e \text{ do } s \text{ od } \mathbf{ok} \rightsquigarrow} \end{array}$$

Static Semantics Rules

$$\begin{array}{c} \frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}} \\[10pt] \frac{U \cup \{y\}; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow W \setminus \{y\}} \\[10pt] \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s_1 \mathbf{ok} \rightsquigarrow W_1 \quad U; V \vdash s_2 \mathbf{ok} \rightsquigarrow W_2}{U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \mathbf{ok} \rightsquigarrow W_1 \cap W_2} \\[10pt] \frac{\text{FV}(e) \subseteq V}{U; V \vdash \text{while } e \text{ do } s \text{ od } \mathbf{ok} \rightsquigarrow} \end{array}$$

Static Semantics Rules

$$\begin{array}{c} \frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}} \\[10pt] \frac{U \cup \{y\}; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow W \setminus \{y\}} \\[10pt] \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s_1 \mathbf{ok} \rightsquigarrow W_1 \quad U; V \vdash s_2 \mathbf{ok} \rightsquigarrow W_2}{U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \mathbf{ok} \rightsquigarrow W_1 \cap W_2} \\[10pt] \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{while } e \text{ do } s \text{ od } \mathbf{ok} \rightsquigarrow} \end{array}$$

Static Semantics Rules

$$\begin{array}{c}
 \frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}} \\
 \frac{U \cup \{y\}; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow W \setminus \{y\}} \\
 \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s_1 \mathbf{ok} \rightsquigarrow W_1 \quad U; V \vdash s_2 \mathbf{ok} \rightsquigarrow W_2}{U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \mathbf{ok} \rightsquigarrow W_1 \cap W_2} \\
 \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{while } e \text{ do } s \text{ od } \mathbf{ok} \rightsquigarrow \emptyset}
 \end{array}$$

Static Semantics Rules

$$\begin{array}{c}
 \frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}} \\
 \frac{U \cup \{y\}; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow W \setminus \{y\}} \\
 \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s_1 \mathbf{ok} \rightsquigarrow W_1 \quad U; V \vdash s_2 \mathbf{ok} \rightsquigarrow W_2}{U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \mathbf{ok} \rightsquigarrow W_1 \cap W_2} \\
 \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{while } e \text{ do } s \text{ od } \mathbf{ok} \rightsquigarrow \emptyset} \\
 \hline
 U; V \vdash s_1; s_2 \mathbf{ok} \rightsquigarrow
 \end{array}$$

Static Semantics Rules

$$\begin{array}{c}
 \frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}} \\
 \frac{U \cup \{y\}; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow W \setminus \{y\}} \\
 \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s_1 \mathbf{ok} \rightsquigarrow W_1 \quad U; V \vdash s_2 \mathbf{ok} \rightsquigarrow W_2}{U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \mathbf{ok} \rightsquigarrow W_1 \cap W_2} \\
 \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{while } e \text{ do } s \text{ od } \mathbf{ok} \rightsquigarrow \emptyset} \\
 \frac{U; V \vdash s_1 \mathbf{ok} \rightsquigarrow W_1}{U; V \vdash s_1; s_2 \mathbf{ok} \rightsquigarrow}
 \end{array}$$

Static Semantics Rules

$$\begin{array}{c}
 \frac{}{U; V \vdash \text{skip } \mathbf{ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \mathbf{ok} \rightsquigarrow \{x\}} \\
 \\
 \frac{U \cup \{y\}; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \mathbf{ok} \rightsquigarrow W \setminus \{y\}} \\
 \\
 \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s_1 \mathbf{ok} \rightsquigarrow W_1 \quad U; V \vdash s_2 \mathbf{ok} \rightsquigarrow W_2}{U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \mathbf{ok} \rightsquigarrow W_1 \cap W_2} \\
 \\
 \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s \mathbf{ok} \rightsquigarrow W}{U; V \vdash \text{while } e \text{ do } s \text{ od } \mathbf{ok} \rightsquigarrow \emptyset} \\
 \\
 \frac{U; V \vdash s_1 \mathbf{ok} \rightsquigarrow W_1 \quad U; (V \cup W_1) \vdash s_2 \mathbf{ok} \rightsquigarrow W_2}{U; V \vdash s_1; s_2 \mathbf{ok} \rightsquigarrow}
 \end{array}$$

Static Semantics Rules

$$\begin{array}{c}
 \frac{}{U; V \vdash \text{skip} \text{ ok} \rightsquigarrow \emptyset} \quad \frac{x \in U \quad \text{FV}(e) \subseteq V}{U; V \vdash x := e \text{ ok} \rightsquigarrow \{x\}} \\
 \\
 \frac{U \cup \{y\}; V \vdash s \text{ ok} \rightsquigarrow W}{U; V \vdash \text{var } y \cdot s \text{ ok} \rightsquigarrow W \setminus \{y\}} \\
 \\
 \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s_1 \text{ ok} \rightsquigarrow W_1 \quad U; V \vdash s_2 \text{ ok} \rightsquigarrow W_2}{U; V \vdash \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi} \text{ ok} \rightsquigarrow W_1 \cap W_2} \\
 \\
 \frac{\text{FV}(e) \subseteq V \quad U; V \vdash s \text{ ok} \rightsquigarrow W}{U; V \vdash \text{while } e \text{ do } s \text{ od} \text{ ok} \rightsquigarrow \emptyset} \\
 \\
 \frac{U; V \vdash s_1 \text{ ok} \rightsquigarrow W_1 \quad U; (V \cup W_1) \vdash s_2 \text{ ok} \rightsquigarrow W_2}{U; V \vdash s_1; s_2 \text{ ok} \rightsquigarrow W_1 \cup W_2}
 \end{array}$$

Dynamic Semantics

We will use **big-step** operational semantics. What are the sets of evaluable expressions and values here?

Dynamic Semantics

We will use **big-step** operational semantics. What are the sets of evaluable expressions and values here?

Evaluable Expressions:

Dynamic Semantics

We will use **big-step** operational semantics. What are the sets of evaluable expressions and values here?

Evaluable Expressions: A pair containing a **statement** to execute and a *state* σ .

Values:

Dynamic Semantics

We will use **big-step** operational semantics. What are the sets of evaluable expressions and values here?

Evaluable Expressions: A pair containing a **statement** to execute and a **state** σ .

Values: The final **state** that results from executing the statement.

States: mutable mappings from states to values.

States

A *state* is a mutable mapping from variables to their values. We use the following notation:

- To *read* a variable x from the state σ , we write $\sigma(x)$.

States

A *state* is a mutable mapping from variables to their values. We use the following notation:

- To *read* a variable x from the state σ , we write $\sigma(x)$.
- To *update* an existing variable x to have value v inside the state σ , we write $(\sigma : x \mapsto v)$.

States

A **state** is a mutable mapping from variables to their values. We use the following notation:

- To **read** a variable x from the state σ , we write $\sigma(x)$.
- To **update** an existing variable x to have value v inside the state σ , we write $(\sigma : x \mapsto v)$.
- To **extend** a state σ with a new, previously undeclared variable x , we write $\sigma \cdot x$. In such a state, $(\sigma \cdot x)(x)$ is undefined.

States

A *state* is a mutable mapping from variables to their values. We use the following notation:

- To *read* a variable x from the state σ , we write $\sigma(x)$.
- To *update* an existing variable x to have value v inside the state σ , we write $(\sigma : x \mapsto v)$.
- To *extend* a state σ with a new, previously undeclared variable x , we write $\sigma \cdot x$. In such a state, $(\sigma \cdot x)(x)$ is undefined.
- To *remove* a variable x from the set of declared variables, we write $(\sigma|_x)$.

States

A *state* is a mutable mapping from variables to their values. We use the following notation:

- To *read* a variable x from the state σ , we write $\sigma(x)$.
- To *update* an existing variable x to have value v inside the state σ , we write $(\sigma : x \mapsto v)$.
- To *extend* a state σ with a new, previously undeclared variable x , we write $\sigma \cdot x$. In such a state, $(\sigma \cdot x)(x)$ is undefined.
- To *remove* a variable x from the set of declared variables, we write $(\sigma|_x)$.
- To exit a local scope for x , returning to the previous scope σ' :

$$\sigma|_x^{\sigma'} = \begin{cases} \sigma|_x & \text{if } x \text{ is undeclared in } \sigma' \\ (\sigma|_x) \cdot x & \text{if } x \text{ is declared but undefined in } \sigma' \\ (\sigma : x \mapsto \sigma'(x)) & \text{if } \sigma'(x) \text{ is defined} \end{cases}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\frac{}{(\sigma, \text{skip}) \Downarrow \sigma}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \qquad \frac{}{(\sigma_1, s_1; s_2) \Downarrow}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2}{(\sigma_1, s_1; s_2) \Downarrow}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \qquad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \qquad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\frac{\frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}}{\sigma \vdash e \Downarrow v} \quad \frac{}{(\sigma, x := e) \Downarrow}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$
$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$
$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{}{(\sigma_1, \text{var } x \cdot s) \Downarrow}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$
$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$
$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c} \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\[10pt] \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\[10pt] \frac{}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow} \\[10pt] \frac{}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow} \end{array}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c} \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\[10pt] \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\[10pt] \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow} \\[10pt] \frac{}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow} \end{array}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c} \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\[10pt] \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\[10pt] \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0 \quad (\sigma_1, s_1) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow} \\[10pt] \frac{}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow} \end{array}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c} \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\[10pt] \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\[10pt] \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0 \quad (\sigma_1, s_1) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\[10pt] \frac{}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow} \end{array}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c} \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\[10pt] \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\[10pt] \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0 \quad (\sigma_1, s_1) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\[10pt] \frac{\sigma_1 \vdash e \Downarrow 0}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow} \end{array}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c} \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\[10pt] \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\[10pt] \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0 \quad (\sigma_1, s_1) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\[10pt] \frac{\sigma_1 \vdash e \Downarrow 0 \quad (\sigma_1, s_2) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \end{array}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c}
 \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\
 \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\
 \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0 \quad (\sigma_1, s_1) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0 \quad (\sigma_1, s_2) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow} \quad \frac{}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow}
 \end{array}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c}
 \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\
 \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\
 \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0 \quad (\sigma_1, s_1) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0 \quad (\sigma_1, s_2) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow} \quad \frac{}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow}
 \end{array}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c}
 \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\
 \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\
 \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0 \quad (\sigma_1, s_1) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0 \quad (\sigma_1, s_2) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow \sigma_1} \quad \frac{}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow}
 \end{array}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c}
 \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\
 \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\
 \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0 \quad (\sigma_1, s_1) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0 \quad (\sigma_1, s_2) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow \sigma_1} \quad \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow}
 \end{array}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c}
 \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\
 \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\
 \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0 \quad (\sigma_1, s_1) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0 \quad (\sigma_1, s_2) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0 \quad (\sigma_1, s) \Downarrow \sigma_2 \quad \sigma_1 \vdash e \Downarrow v \quad v \neq 0}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow \sigma_1 \quad (\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow}
 \end{array}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c}
 \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\
 \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\
 \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0 \quad (\sigma_1, s_1) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0 \quad (\sigma_1, s_2) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow \sigma_1} \quad \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0 \quad (\sigma_1, s) \Downarrow \sigma_2 \quad (\sigma_2, \text{while } e \text{ do } s \text{ od}) \Downarrow \sigma_3}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow}
 \end{array}$$

Evaluation Rules

We will assume we have defined a relation $\sigma \vdash e \Downarrow v$ for **arithmetic expressions**, much like in the previous lecture.

$$\begin{array}{c}
 \frac{}{(\sigma, \text{skip}) \Downarrow \sigma} \quad \frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3} \\
 \frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)} \quad \frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}} \\
 \frac{\sigma_1 \vdash e \Downarrow v \quad v \neq 0 \quad (\sigma_1, s_1) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0 \quad (\sigma_1, s_2) \Downarrow \sigma_2}{(\sigma_1, \text{if } e \text{ then } s_1 \text{ else } s_2 \text{ fi}) \Downarrow \sigma_2} \\
 \frac{\sigma_1 \vdash e \Downarrow 0 \quad \sigma_1 \vdash e \Downarrow v \quad v \neq 0}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow \sigma_1} \quad \frac{(\sigma_1, s) \Downarrow \sigma_2 \quad (\sigma_2, \text{while } e \text{ do } s \text{ od}) \Downarrow \sigma_3}{(\sigma_1, \text{while } e \text{ do } s \text{ od}) \Downarrow \sigma_3}
 \end{array}$$

Alternative declaration semantics

What should happen when an uninitialised variable is used?

$$(\sigma \cdot y, \mathbf{var} \ x \cdot y := x + 1) \Downarrow ??$$

Alternative declaration semantics

What should happen when an uninitialised variable is used?

$$(\sigma \cdot y, \mathbf{var} \ x \cdot y := x + 1) \Downarrow ??$$

$$\frac{\frac{???}{(\sigma \cdot y \cdot x, y := x + 1) \Downarrow ??}}{(\sigma \cdot y, \mathbf{var} \ x \cdot y := x + 1) \Downarrow ??}$$

We can't apply the assignment rule here, because in the state $\sigma \cdot y \cdot x$, $\sigma(x)$ is undefined.

Alternative declaration semantics

Crash and burn: $(\sigma \cdot y, \text{var } x \cdot y := x + 1) \Downarrow$

Alternative declaration semantics

Crash and burn: $(\sigma \cdot y, \mathbf{var} \ x \cdot y := x + 1) \Downarrow$

$$\frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \mathbf{var} \ x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}}$$

Default value: $(\sigma \cdot y, \mathbf{var} \ x \cdot y := x + 1) \Downarrow (\sigma \cdot y) : y \mapsto 1$

$$\frac{((\sigma_1 \cdot x) : x \mapsto 0, s) \Downarrow \sigma_2}{(\sigma_1, \mathbf{var} \ x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}}$$

Alternative declaration semantics

Crash and burn: $(\sigma \cdot y, \text{var } x \cdot y := x + 1) \Downarrow$

$$\frac{(\sigma_1 \cdot x, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}}$$

Default value: $(\sigma \cdot y, \text{var } x \cdot y := x + 1) \Downarrow (\sigma \cdot y) : y \mapsto 1$

$$\frac{((\sigma_1 \cdot x) : x \mapsto 0, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}}$$

Junk data: $(\sigma \cdot y, \text{var } x \cdot y := x + 1) \Downarrow (\sigma \cdot y) : y \mapsto 3$ (or 4, or whatever we want...)

$$\frac{((\sigma_1 \cdot x) : x \mapsto n, s) \Downarrow \sigma_2}{(\sigma_1, \text{var } x \cdot s) \Downarrow \sigma_2|_x^{\sigma_1}}$$

Hoare Logic

For a taste of *axiomatic semantics*, let's define a *Hoare Logic* for TinyImp (without **var**). We write a *Hoare triple* judgement as:

$$\{\varphi\} \; s \; \{\psi\}$$

Where φ and ψ are logical formulae about states, called *assertions*, and s is a statement. This triple states that if the statement s successfully evaluates from a starting state satisfying the *precondition* φ , then the final state will satisfy the *postcondition* ψ :

$$\varphi(\sigma) \wedge (\sigma, s) \Downarrow \sigma' \Rightarrow \psi(\sigma')$$

Proving Hoare Triples

To prove a Hoare triple like:

```
{True}  
i := 0;  
m := 1;  
while  $i \neq N$  do  
     $i := i + 1$ ;  
     $m := m \times i$   
od  
{ $m = N!$ }
```

We *could* prove this using the operational semantics. This is cumbersome, and requires an induction to deal with the **while** loop. Instead, we'll define a set of rules to **prove Hoare triples directly** (called *a proof calculus*).

Hoare Rules

$$\frac{}{(\sigma, \text{skip}) \Downarrow \sigma}$$

$$\frac{}{\{\varphi\} \text{ skip}}$$

$$\frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$

$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)}$$

Hoare Rules

$$\frac{}{(\sigma, \text{skip}) \Downarrow \sigma}$$

$$\frac{}{\{\varphi\} \text{ skip } \{\varphi\}}$$

$$\frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$

$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)}$$

Hoare Rules

$$\overline{(\sigma, \text{skip}) \Downarrow \sigma}$$

$$\overline{\{\varphi\} \text{ skip } \{\varphi\}}$$

$$\frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$

$$\frac{}{\{\varphi\} s_1; s_2}$$

$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)}$$

Hoare Rules

$$\overline{(\sigma, \text{skip}) \Downarrow \sigma}$$

$$\overline{\{\varphi\} \text{ skip } \{\varphi\}}$$

$$\frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)}$$

Hoare Rules

$$\overline{(\sigma, \text{skip}) \Downarrow \sigma}$$

$$\overline{\{\varphi\} \text{ skip } \{\varphi\}}$$

$$\frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)}$$

$$\frac{}{x := e \{\varphi\}}$$

Hoare Rules

$$\overline{(\sigma, \text{skip}) \Downarrow \sigma}$$

$$\overline{\{\varphi\} \text{ skip } \{\varphi\}}$$

$$\frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)}$$

$$\overline{\{\varphi[x := e]\} x := e \{\varphi\}}$$

Continuing on, we can get rules for if, and while with a *loop invariant*:

$$\overline{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\overline{\{\varphi\} \text{ while } e \text{ do } s \text{ od}}$$

Hoare Rules

$$\overline{(\sigma, \text{skip}) \Downarrow \sigma}$$

$$\overline{\{\varphi\} \text{ skip } \{\varphi\}}$$

$$\frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)}$$

$$\overline{\{\varphi[x := e]\} x := e \{\varphi\}}$$

Continuing on, we can get rules for if, and while with a *loop invariant*:

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\overline{\{\varphi\} \text{ while } e \text{ do } s \text{ od}}$$

Hoare Rules

$$\overline{(\sigma, \text{skip}) \Downarrow \sigma}$$

$$\overline{\{\varphi\} \text{ skip } \{\varphi\}}$$

$$\frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)}$$

$$\overline{\{\varphi[x := e]\} x := e \{\varphi\}}$$

Continuing on, we can get rules for if, and while with a *loop invariant*:

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\overline{\{\varphi\} \text{ while } e \text{ do } s \text{ od}}$$

Hoare Rules

$$\overline{(\sigma, \text{skip}) \Downarrow \sigma}$$

$$\overline{\{\varphi\} \text{ skip } \{\varphi\}}$$

$$\frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)}$$

$$\overline{\{\varphi[x := e]\} x := e \{\varphi\}}$$

Continuing on, we can get rules for if, and while with a *loop invariant*:

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od}}$$

Hoare Rules

$$\overline{(\sigma, \text{skip}) \Downarrow \sigma}$$

$$\overline{\{\varphi\} \text{ skip } \{\varphi\}}$$

$$\frac{(\sigma_1, s_1) \Downarrow \sigma_2 \quad (\sigma_2, s_2) \Downarrow \sigma_3}{(\sigma_1, s_1; s_2) \Downarrow \sigma_3}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\sigma \vdash e \Downarrow v}{(\sigma, x := e) \Downarrow (\sigma : x \mapsto v)}$$

$$\overline{\{\varphi[x := e]\} x := e \{\varphi\}}$$

Continuing on, we can get rules for if, and while with a *loop invariant*:

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

Consequence

There is one more rule, called the *rule of consequence*, that we need to insert ordinary logical reasoning into our Hoare logic proofs:

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} \text{ s } \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} \text{ s } \{\psi\}}$$

Consequence

There is one more rule, called the *rule of consequence*, that we need to insert ordinary logical reasoning into our Hoare logic proofs:

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

This is the only rule that is **not** directed entirely by syntax. This means a Hoare logic proof need not look like a derivation tree. Instead we can sprinkle assertions through our program and specially note uses of the consequence rule.

Factorial Example

Let's verify the Factorial program using our Hoare rules:

{True}

$i := 0;$
 $m := 1;$

while $i \neq N$ **do**

$i := i + 1;$

$m := m \times i$

od

{ $m = N!$ }

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

{True}

$i := 0;$
 $m := 1;$

while $i \neq N$ **do**

$i := i + 1;$

$m := m \times i$

od $\{m = i! \wedge i = N\}$
 $\{m = N!\}$

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{\text{True}\}$

$i := 0;$

$m := 1;$

$\{m = i!\}$

while $i \neq N$ **do**

$i := i + 1;$

$m := m \times i$

od $\{m = i! \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{\text{True}\}$

$i := 0;$

$m := 1;$

$\{m = i!\}$

while $i \neq N$ **do**

$i := i + 1;$

$m := m \times i$

$\{m = i!\}$

od $\{m = i! \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{\text{True}\}$

$i := 0;$

$m := 1;$

$\{m = i!\}$

while $i \neq N$ **do** $\{m = i! \wedge i \neq N\}$

$i := i + 1;$

$m := m \times i$

$\{m = i!\}$

od $\{m = i! \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```

{True}
    i := 0;
    m := 1;
    {m = i!}
    while i ≠ N do {m = i! ∧ i ≠ N}
        i := i + 1;
        {m × i = i!}
        m := m × i
        {m = i!}
    od {m = i! ∧ i = N}
    {m = N!}
    
```

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

$\{\text{True}\}$

$i := 0;$

$m := 1;$

$\{m = i!\}$

while $i \neq N$ **do** $\{m = i! \wedge i \neq N\}$

$\{m \times (i + 1) = (i + 1)!\}$

$i := i + 1;$

$\{m \times i = i!\}$

$m := m \times i$

$\{m = i!\}$

od $\{m = i! \wedge i = N\}$

$\{m = N!\}$

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```
{True}
  i := 0;
  m := 1;
{m = i!}
while i ≠ N do {m = i! ∧ i ≠ N}
  {m × (i + 1) = (i + 1)!}
  i := i + 1;
  {m × i = i!}
  m := m × i
  {m = i!}
od {m = i! ∧ i = N}
{m = N!}
```

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```

{True}
  i := 0;
  m := 1; {m = i!}
{m = i!}
while i ≠ N do {m = i! ∧ i ≠ N}
  {m × (i + 1) = (i + 1)!}
  i := i + 1;
  {m × i = i!}
  m := m × i
  {m = i!}
od {m = i! ∧ i = N}
{m = N!}
    
```

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```

{True}
  i := 0;
{1 = i!} m := 1; {m = i!}
{m = i!}
while i ≠ N do {m = i! ∧ i ≠ N}
  {m × (i + 1) = (i + 1)!}
  i := i + 1;
  {m × i = i!}
  m := m × i
od {m = i! ∧ i = N}
{m = N!}
    
```

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```

{True}
  i := 0; {1 = i!}
{1 = i!} m := 1; {m = i!}
{m = i!}
while i ≠ N do {m = i! ∧ i ≠ N}
  {m × (i + 1) = (i + 1)!}
  i := i + 1;
  {m × i = i!}
  m := m × i
  {m = i!}
od {m = i! ∧ i = N}
{m = N!}
    
```

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

note: $(i + 1)! = i! \times (i + 1)$

Factorial Example

Let's verify the Factorial program using our Hoare rules:

```

{True}
{1 = 0!} i := 0; {1 = i!}
{1 = i!} m := 1; {m = i!}
{m = i!}
while i ≠ N do {m = i! ∧ i ≠ N}
  {m × (i + 1) = (i + 1)!}
  i := i + 1;
  {m × i = i!}
  m := m × i
od {m = i! ∧ i = N}
{m = N!}
    
```

$$\frac{\{\varphi \wedge e\} s_1 \{\psi\} \quad \{\varphi \wedge \neg e\} s_2 \{\psi\}}{\{\varphi\} \text{ if } e \text{ then } s_1 \text{ else } s_2 \text{ fi } \{\psi\}}$$

$$\frac{}{\{\varphi[x := e]\} x := e \{\varphi\}}$$

$$\frac{\{\varphi \wedge e\} s \{\varphi\}}{\{\varphi\} \text{ while } e \text{ do } s \text{ od } \{\varphi \wedge \neg e\}}$$

$$\frac{\{\varphi\} s_1 \{\alpha\} \quad \{\alpha\} s_2 \{\psi\}}{\{\varphi\} s_1; s_2 \{\psi\}}$$

$$\frac{\varphi \Rightarrow \alpha \quad \{\alpha\} s \{\beta\} \quad \beta \Rightarrow \psi}{\{\varphi\} s \{\psi\}}$$

note: $(i + 1)! = i! \times (i + 1)$